

```
##### Team IOTA #####

#importing libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import warnings
warnings.filterwarnings('ignore')
```

from google.colab import files

Choose your file from our device to import
uploaded = files.upload()

Choose Files

model-1.csv

model-1.csv(text/csv) - 21553 bytes, last modified: 9/25/2025 - 100% done
Saving model-1.csv to model-1.csv

#importing the dataset
data = pd.read_csv('model-1.csv')
data.head()

	Temparature	Humidity	Moisture	Soil_Type	Crop_Type	Nitrogen	Potassium	Phos
0	20	83	26	Clayey	rice	90	49	
1	25	84	32	Loamy	rice	66	59	
2	33	64	50	Loamy	Wheat	41	0	
3	34	65	54	Loamy	Wheat	38	0	
4	38	72	51	Loamy	Wheat	39	0	

Next steps: [Generate code with data](#) [New interactive sheet](#)

```
data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 552 entries, 0 to 551
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Temparature     552 non-null   int64
1   Humidity        552 non-null   int64
2   Moisture        552 non-null   int64
3   Soil_Type       552 non-null   object
```

```
4 Crop_Type      552 non-null    object
5 Nitrogen       552 non-null    int64
6 Potassium      552 non-null    int64
7 Phosphorous    552 non-null    int64
8 Fertilizer     552 non-null    object
dtypes: int64(6), object(3)
memory usage: 38.9+ KB
```

```
#changing the column names
data.rename(columns={'Humidity ':'Humidity',
                    'Soil Type':'Soil_Type',
                    'Crop Type':'Crop_Type',
                    'Fertilizer Name':'Fertilizer'},
            inplace=True)
```

```
#checking unique values
data.nunique()
```

	0
Temparature	21
Humidity	34
Moisture	41
Soil_Type	5
Crop_Type	17
Nitrogen	47
Potassium	31
Phosphorous	36
Fertilizer	14

dtype: int64

```
#checking for null values
data.isna().sum()
```

	0
Temperature	0
Humidity	0
Moisture	0
Soil_Type	0
Crop_Type	0
Nitrogen	0
Potassium	0
Phosphorous	0
Fertilizer	0

dtype: int64

```
data['Fertilizer'].unique()
```

```
array(['Urea', 'TSP', 'Superphosphate', 'Potassium sulfate.',
      'Potassium chloride', 'DAP', '28-28', '20-20', '17-17-17',
      '15-15-15', '14-35-14', '14-14-14', '10-26-26', '10-10-2010'],
      dtype=object)
```

```
# Updated mapping dictionary for replacement
fertilizer_map = {
    "10-10-10": "Urea + TSP + Potassium chloride",
    "10-10-2010": "Urea + TSP + Potassium chloride",
    "10-26-26": "Urea + TSP + Potassium sulfate",
    "14-14-14": "Urea + TSP + Potassium chloride",
    "14-35-14": "Urea + TSP + Potassium chloride",
    "15-15-15": "Urea + TSP + Potassium chloride",
    "17-17-17": "Urea + TSP + Potassium chloride",
    "28-28": "Urea + TSP",
    "20-20": "Urea + TSP"
}

# Replace values in Fertilizer column
data["Fertilizer"] = data["Fertilizer"].replace(fertilizer_map)

# Save back to CSV
data.to_csv("model-1.csv", index=False)

print("Replacement done, file updated.")
```

Replacement done, file updated.

```
data['Fertilizer'].unique()
```

```
array(['Urea', 'TSP', 'Superphosphate', 'Potassium sulfate.',  
      'Potassium chloride', 'DAP', 'Urea + TSP',  
      'Urea + TSP + Potassium chloride',  
      'Urea + TSP + Potassium sulfate'], dtype=object)
```

```
# Replace all occurrences of "TSP (or Superphosphate)" with "TSP"  
data["Fertilizer"] = data["Fertilizer"].str.replace(  
    "TSP \\\(or Superphosphate\\)", "TSP", regex=True  
)
```

```
# Save back to CSV  
data.to_csv("model-1.csv", index=False)
```

```
print(" All 'TSP (or Superphosphate)' replaced with 'TSP'")
```

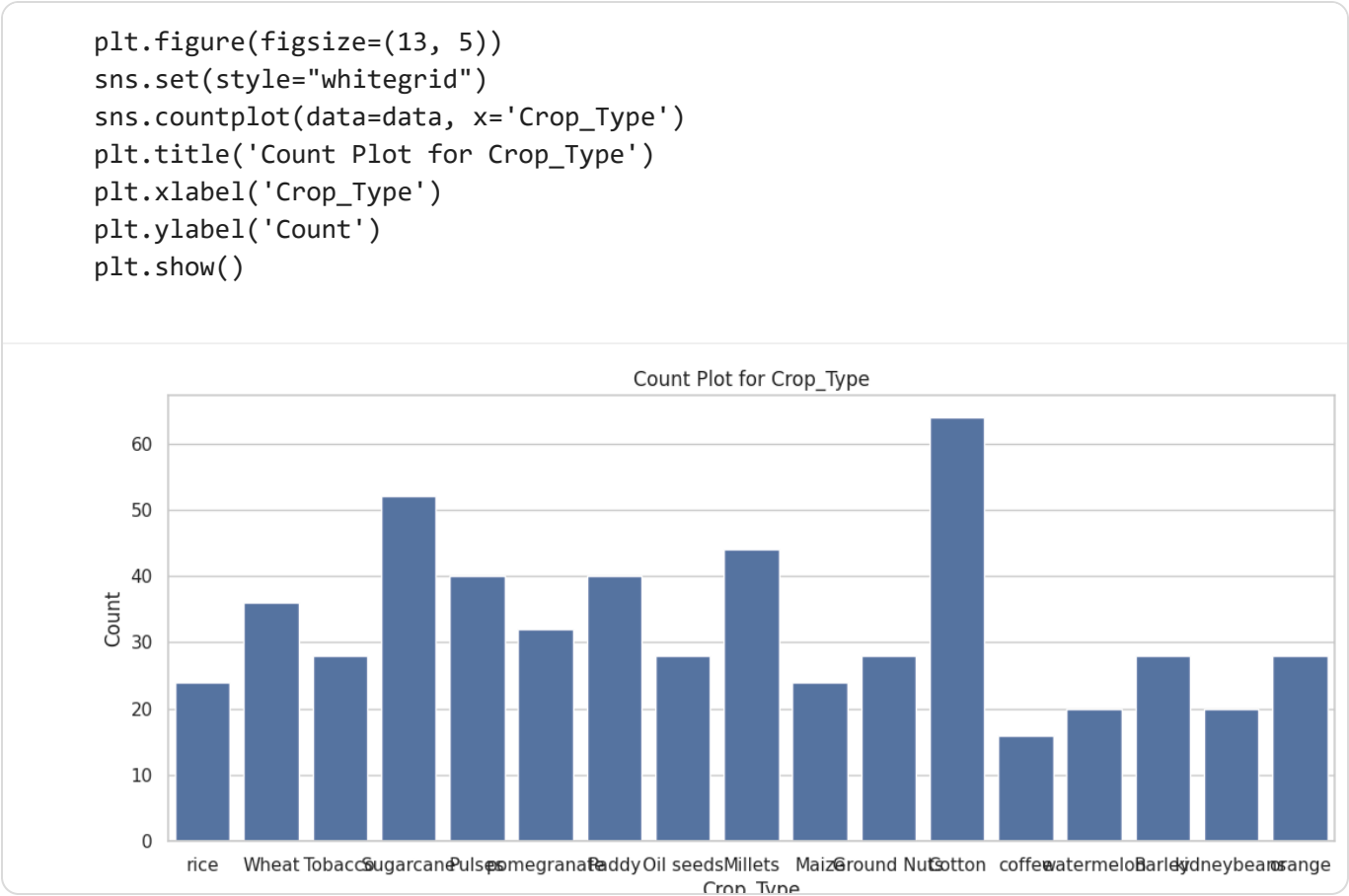
```
All 'TSP (or Superphosphate)' replaced with 'TSP'
```

```
data['Fertilizer'].unique()
```

```
array(['Urea', 'TSP', 'Superphosphate', 'Potassium sulfate.',  
      'Potassium chloride', 'DAP', 'Urea + TSP',  
      'Urea + TSP + Potassium chloride',  
      'Urea + TSP + Potassium sulfate'], dtype=object)
```

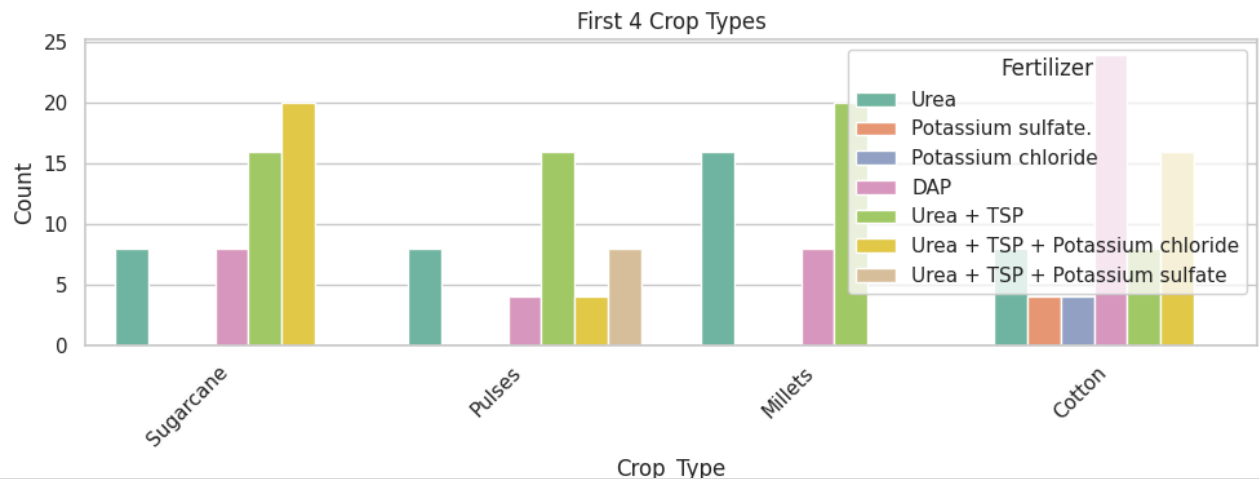
```
#statistical parameters  
data.describe(include='all')
```

	Temperature	Humidity	Moisture	Soil_Type	Crop_Type	Nitrogen	Potas
count	552.000000	552.000000	552.000000	552	552	552.000000	552.00
unique	NaN	NaN	NaN	5	17	NaN	
top	NaN	NaN	NaN	Loamy	Cotton	NaN	
freq	NaN	NaN	NaN	192	64	NaN	
mean	28.630435	64.557971	42.840580	NaN	NaN	28.521739	10.14
std	5.088082	11.880236	11.507275	NaN	NaN	29.121989	13.45
min	0.000000	50.000000	25.000000	NaN	NaN	0.000000	0.00
25%	26.000000	54.000000	33.000000	NaN	NaN	10.000000	0.00
50%	29.000000	62.000000	41.000000	NaN	NaN	15.000000	0.00
75%	32.000000	68.000000	51.000000	NaN	NaN	37.000000	18.00
max	38.000000	95.000000	65.000000	NaN	NaN	126.000000	59.00



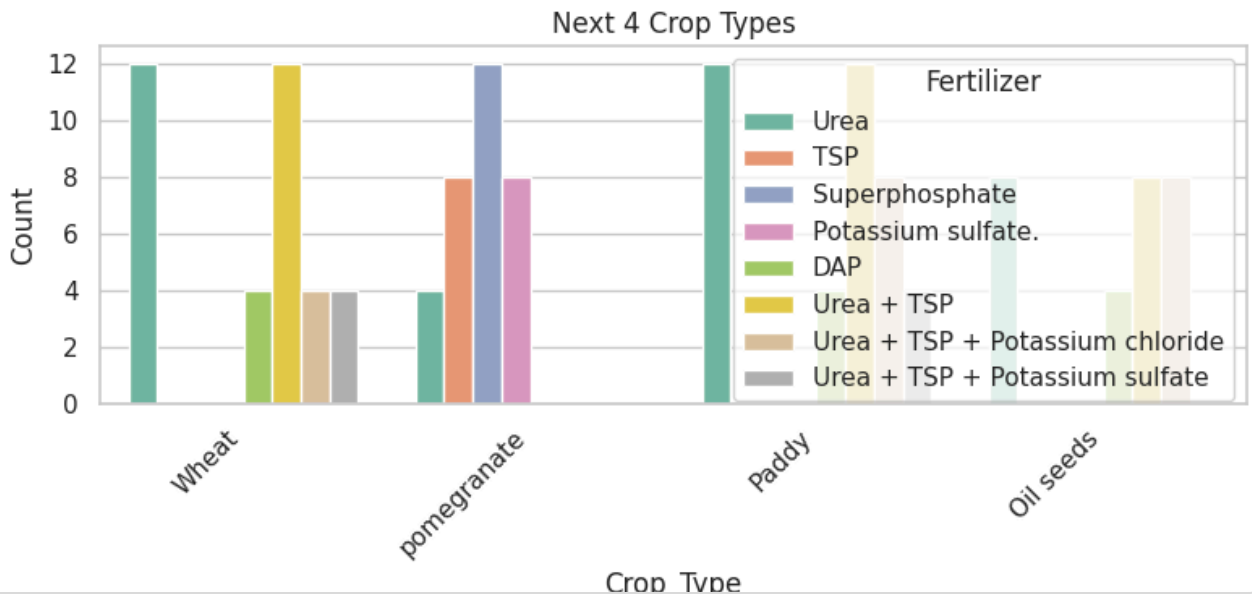
```
#first 4 crop types
part1_data = data[data['Crop_Type'].isin(data['Crop_Type'].value_counts().index[:4])]

# Create the first countplot
plt.figure(figsize=(10, 4))
sns.set(style="whitegrid")
sns.countplot(data=part1_data, x='Crop_Type', hue='Fertilizer', width=0.8, palette=
plt.title('First 4 Crop Types')
plt.xlabel('Crop_Type')
plt.ylabel('Count')
plt.legend(title='Fertilizer')
plt.xticks(rotation=45, horizontalalignment='right')
plt.tight_layout()
plt.show()
```



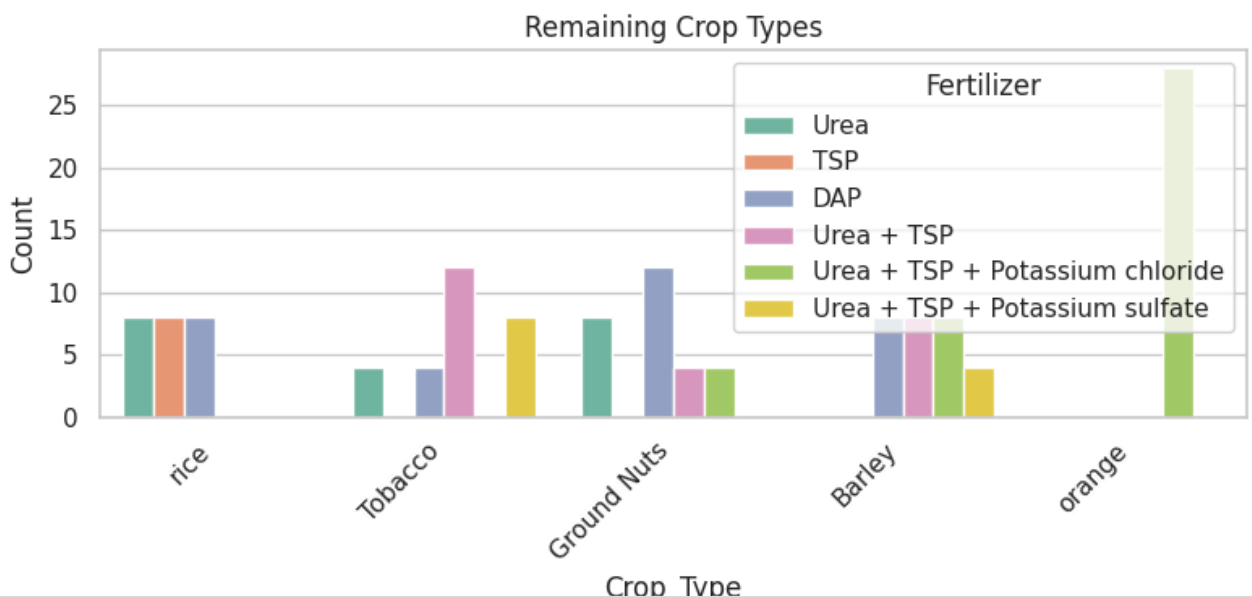
```
# Split the data into three parts: next 4 crop types
part2_data = data[data['Crop_Type'].isin(data['Crop_Type'].value_counts().index[4:])]

# Create the second countplot
plt.figure(figsize=(8, 4))
sns.set(style="whitegrid")
sns.countplot(data=part2_data, x='Crop_Type', hue='Fertilizer', width=0.8, palette=
plt.title('Next 4 Crop Types')
plt.xlabel('Crop_Type')
plt.ylabel('Count')
plt.legend(title='Fertilizer')
plt.xticks(rotation=45, horizontalalignment='right')
plt.tight_layout()
plt.show()
```



```
# Split the data into three parts: remaining crop types
part3_data = data[data['Crop_Type'].isin(data['Crop_Type'].value_counts().index[8:

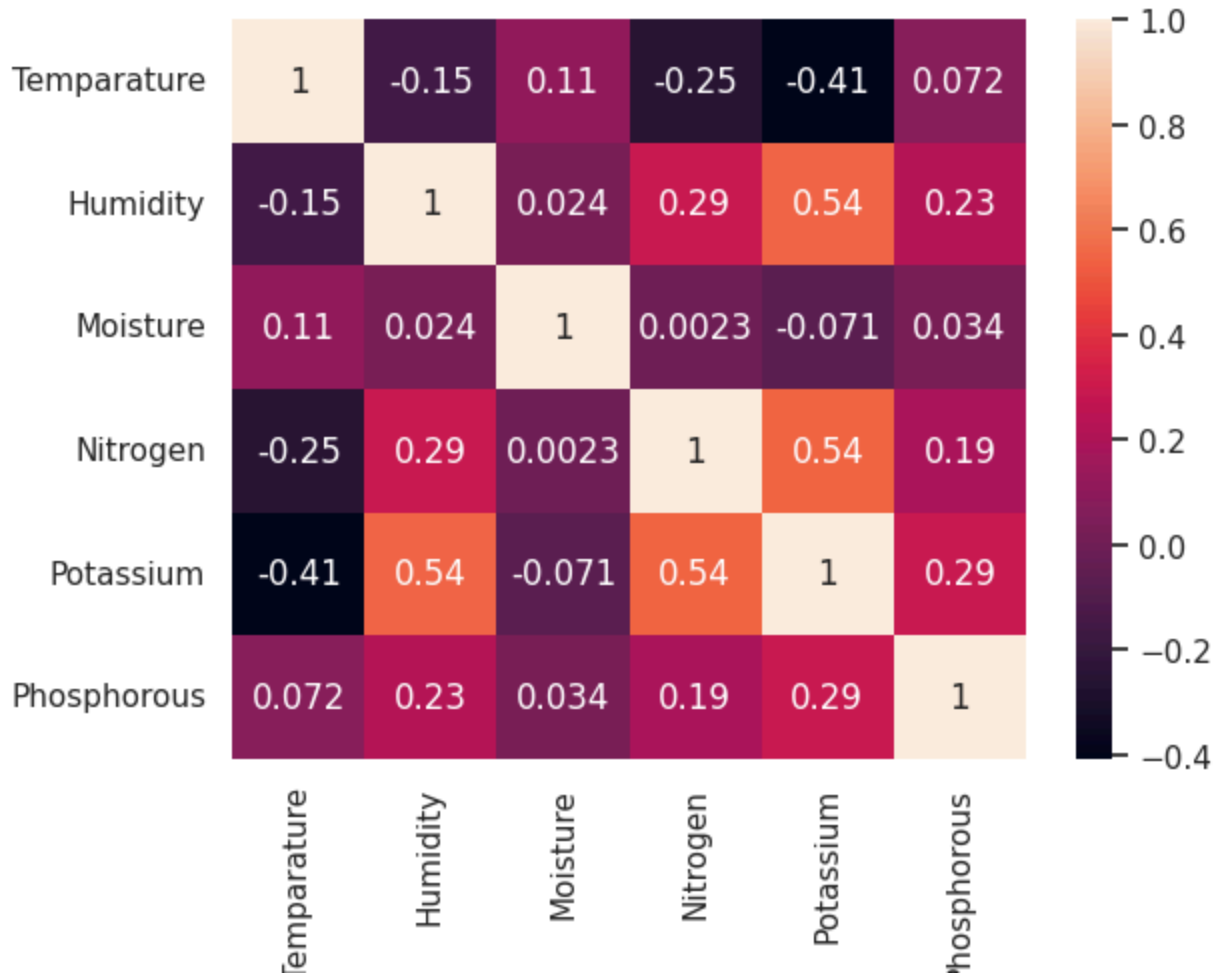
# Create the third countplot
plt.figure(figsize=(8, 4))
sns.set(style="whitegrid")
sns.countplot(data=part3_data, x='Crop_Type', hue='Fertilizer', width=0.8, palette=
plt.title('Remaining Crop Types')
plt.xlabel('Crop_Type')
plt.ylabel('Count')
plt.legend(title='Fertilizer')
plt.xticks(rotation=45, horizontalalignment='right')
plt.tight_layout()
plt.show()
```



```
# this plot is provides insights into how different crop types are distributed ba
# The x-axis represents the different crop types.
# The y-axis represents the count (the number of occurrences) of each crop type ir
```

```
sns.heatmap(data.select_dtypes(include=[np.number]).corr(), annot=True)
```

<Axes: >



```
#there is no strong correlation between any input attributes
```

```
#encoding the labels for categorical variables
from sklearn.preprocessing import LabelEncoder
#transform non-numeric data into a numeric format
```

```
#encoding Soil Type variable
encode_soil = LabelEncoder()

#fitting the label encoder
data.Soil_Type = encode_soil.fit_transform(data.Soil_Type)

#creating the DataFrame
```

```
Soil_Type = pd.DataFrame(zip(encode_soil.classes_,encode_soil.transform(encode_soil.classes_)))
Soil_Type = Soil_Type.set_index('Original')
Soil_Type
```

	Encoded	
Original		
Black	0	
Clayey	1	
Loamy	2	
Red	3	
Sandy	4	

Next steps:

[Generate code with Soil_Type](#)[New interactive sheet](#)

```
#encoding Crop Type variable
encode_crop = LabelEncoder()

#fitting the label encoder
data.Crop_Type = encode_crop.fit_transform(data.Crop_Type)

#creating the DataFrame
Crop_Type = pd.DataFrame(zip(encode_crop.classes_,encode_crop.transform(encode_crop.classes_)))
Crop_Type = Crop_Type.set_index('Original')
Crop_Type
```

	Encoded	
Original		
Barley	0	
Cotton	1	
Ground Nuts	2	
Maize	3	
Millet	4	
Oil seeds	5	
Paddy	6	
Pulses	7	
Sugarcane	8	
Tobacco	9	
Wheat	10	
coffee	11	
kidneybeans	12	
orange	13	
pomegranate	14	
rice	15	
watermelon	16	

Next steps:

[Generate code with Crop_Type](#)[New interactive sheet](#)

```

#encoding Fertilizer variable
encode_fert = LabelEncoder()

#fitting the label encoder
data.Fertilizer = encode_fert.fit_transform(data.Fertilizer)

#creating the DataFrame
Fertilizer = pd.DataFrame(zip(encode_fert.classes_, encode_fert.transform(encode_fert.classes_)))
Fertilizer = Fertilizer.set_index('Original')
Fertilizer

```

Original	Encoded	
DAP	0	
Potassium chloride	1	
Potassium sulfate.	2	
Superphosphate	3	
TSP	4	
Urea	5	
Urea + TSP	6	
Urea + TSP + Potassium chloride	7	
Urea + TSP + Potassium sulfate	8	

Next steps:

[Generate code with Fertilizer](#)[New interactive sheet](#)

```
#splitting the data into input and output variables
X = data.drop(['Fertilizer'], axis=1)
y = data['Fertilizer']
```

```
#splitting the data into training and testing sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_s
```

```
#Feature Scaling
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
#Building the model
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
```

▼ RandomForestClassifier ⓘ ?

```
RandomForestClassifier(random_state=42)
```

```
#predicting the test set results
y_pred = rf.predict(X_test)
```

```
#evaluating the model
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	16
2	1.00	1.00	1.00	1
3	1.00	1.00	1.00	1
4	1.00	1.00	1.00	5
5	1.00	1.00	1.00	24
6	1.00	1.00	1.00	32
7	1.00	1.00	1.00	23
8	1.00	1.00	1.00	9
accuracy			1.00	111
macro avg	1.00	1.00	1.00	111
weighted avg	1.00	1.00	1.00	111

Confusion Matrix:

```
[[16  0  0  0  0  0  0  0]
 [ 0  1  0  0  0  0  0  0]
 [ 0  0  1  0  0  0  0  0]
 [ 0  0  0  5  0  0  0  0]
 [ 0  0  0  0 24  0  0  0]
 [ 0  0  0  0  0 32  0  0]
 [ 0  0  0  0  0  0 23  0]
 [ 0  0  0  0  0  0  0  9]]
```

```
acc = [] # TEST
model = []
acc1=[] # TRIAN
```

```
# Decision Tree Classifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report
from sklearn import metrics
from sklearn import tree
import warnings
warnings.filterwarnings('ignore')
```

```
ds = DecisionTreeClassifier(criterion="entropy", random_state=2, max_depth=5)
```

```

ds.fit(X_train, y_train)

# --- Testing predictions ---
y_pred_test = ds.predict(X_test)
test_acc = metrics.accuracy_score(y_test, y_pred_test)
acc.append(test_acc)

# --- Training predictions ---
y_pred_train = ds.predict(X_train)
train_acc = metrics.accuracy_score(y_train, y_pred_train)
acc1.append(train_acc)

# Store model name
model.append('Decision Tree')

# Print accuracies
print("Decision Tree's Accuracy → Test:", test_acc*100, "Train:", train_acc*100)

# Classification report on TEST predictions only
print("\nClassification Report (Test Data):\n")
print(classification_report(y_test, y_pred_test))

```

Decision Tree's Accuracy → Test: 97.2972972972973 Train: 97.95918367346938

Classification Report (Test Data):

	precision	recall	f1-score	support
0	1.00	1.00	1.00	16
2	1.00	1.00	1.00	1
3	0.50	1.00	0.67	1
4	0.71	1.00	0.83	5
5	1.00	0.88	0.93	24
6	1.00	1.00	1.00	32
7	1.00	1.00	1.00	23
8	1.00	1.00	1.00	9
accuracy			0.97	111
macro avg	0.90	0.98	0.93	111
weighted avg	0.98	0.97	0.98	111

```

from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import classification_report, accuracy_score

# Train model
NaiveBayes = GaussianNB()
NaiveBayes.fit(X_train, y_train)

# --- Testing predictions ---
y_pred_test = NaiveBayes.predict(X_test)

```

```

test_acc = accuracy_score(y_test, y_pred_test)
acc.append(test_acc)

# --- Training predictions ---
y_pred_train = NaiveBayes.predict(X_train)
train_acc = accuracy_score(y_train, y_pred_train)
acc1.append(train_acc)

# Store model name
model.append('Naive Bayes')

# Print accuracies
print("Naive Bayes Accuracy → Test:", test_acc, "Train:", train_acc)

# Classification report (only for TEST data)
print("\nClassification Report (Test Data):\n")
print(classification_report(y_test, y_pred_test))

```

Naive Bayes Accuracy → Test: 0.6936936936936937 Train: 0.6417233560090703

Classification Report (Test Data):

	precision	recall	f1-score	support
0	0.40	0.12	0.19	16
2	1.00	1.00	1.00	1
3	0.33	1.00	0.50	1
4	0.67	0.80	0.73	5
5	1.00	0.21	0.34	24
6	0.54	1.00	0.70	32
7	1.00	1.00	1.00	23
8	1.00	1.00	1.00	9
accuracy			0.69	111
macro avg	0.74	0.77	0.68	111
weighted avg	0.76	0.69	0.64	111

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score

# Train model
RF = RandomForestClassifier(n_estimators=20, random_state=0)
RF.fit(X_train, y_train)

# --- Test predictions ---
y_pred_test = RF.predict(X_test)
test_acc = accuracy_score(y_test, y_pred_test)
acc.append(test_acc)

# --- Train predictions ---
y_pred_train = RF.predict(X_train)

```

```

train_acc = accuracy_score(y_train, y_pred_train)
acc1.append(train_acc)

# Store model name
model.append('RF')

# Print accuracies
print("RF's Accuracy → Test:", test_acc, "Train:", train_acc)

# Classification report only for TEST set
print("\nClassification Report (Test Data):\n")
print(classification_report(y_test, y_pred_test))

```

RF's Accuracy → Test: 1.0 Train: 1.0

Classification Report (Test Data):

	precision	recall	f1-score	support
0	1.00	1.00	1.00	16
2	1.00	1.00	1.00	1
3	1.00	1.00	1.00	1
4	1.00	1.00	1.00	5
5	1.00	1.00	1.00	24
6	1.00	1.00	1.00	32
7	1.00	1.00	1.00	23
8	1.00	1.00	1.00	9
accuracy			1.00	111
macro avg	1.00	1.00	1.00	111
weighted avg	1.00	1.00	1.00	111

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score

# Train model
LogReg = LogisticRegression(random_state=2, max_iter=1000) # added max_iter for s
LogReg.fit(X_train, y_train)

# --- Test predictions ---
y_pred_test = LogReg.predict(X_test)
test_acc = accuracy_score(y_test, y_pred_test)
acc.append(test_acc)

# --- Train predictions ---
y_pred_train = LogReg.predict(X_train)
train_acc = accuracy_score(y_train, y_pred_train)
acc1.append(train_acc)

# Store model name
model.append('Logistic Regression')

```

```
# Print accuracies
print("Logistic Regression's Accuracy → Test:", test_acc, "Train:", train_acc)

# Classification report only for TEST set
print("\nClassification Report (Test Data):\n")
print(classification_report(y_test, y_pred_test))
```

Logistic Regression's Accuracy → Test: 0.8648648648648649 Train: 0.8888888888888888

Classification Report (Test Data):

	precision	recall	f1-score	support
0	0.83	0.94	0.88	16
2	0.00	0.00	0.00	1
3	0.50	1.00	0.67	1
4	0.80	0.80	0.80	5
5	1.00	0.88	0.93	24
6	1.00	0.97	0.98	32
7	0.70	1.00	0.82	23
8	1.00	0.11	0.20	9
accuracy			0.86	111
macro avg	0.73	0.71	0.66	111
weighted avg	0.89	0.86	0.84	111

```
from sklearn.model_selection import cross_val_score
```

```
score = cross_val_score(RF,data,data.Fertilizer,cv=5)
print("Cross-validation score of RF is:",score)
score = cross_val_score(ds, data, data.Fertilizer,cv=5)
print("Cross-validation score of ds is:",score)
score = cross_val_score(LogReg,data,data.Fertilizer,cv=5)
print("Cross-validation score of LogReg is:",score)
score = cross_val_score(NaiveBayes,data,data.Fertilizer,cv=5)
print("Cross-validation score of NaiveBayes is:",score)
```

```
Cross-validation score of RF is: [1. 1. 1. 1. 1.]
Cross-validation score of ds is: [1. 1. 1. 1. 1.]
Cross-validation score of LogReg is: [1. 0.97297297 0.97272727 0.99090909 0.99090909]
Cross-validation score of NaiveBayes is: [1. 1. 1. 1. 1.]
```

```
print("Length of model:", len(model))
print("Length of acc:", len(acc))
print("Models:", model)
print("Accuracies:", acc)
```

```
Length of model: 4
Length of acc: 4
Models: ['Decision Tree', 'Naive Bayes', 'RF', 'Logistic Regression']
Accuracies: [0.972972972972973, 0.6936936936936937, 1.0, 0.8648648648648649]
```

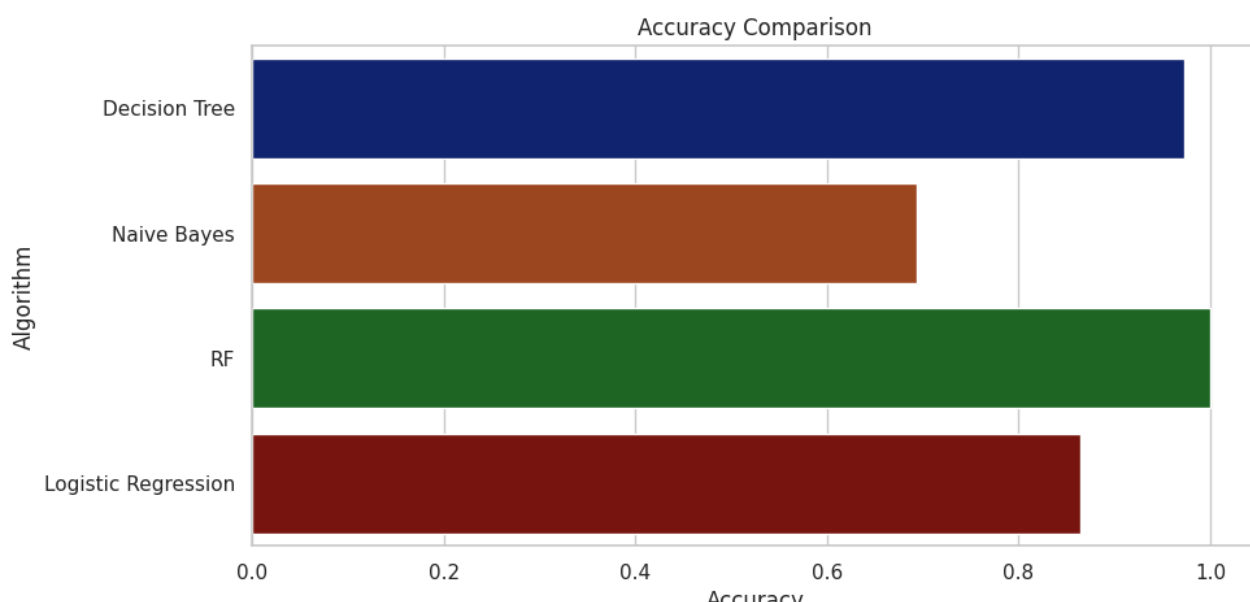
```
min_len = min(len(model), len(acc))
model = model[:min_len]
acc = acc[:min_len]
```

```
print("Length of model:", len(model))
print("Length of acc:", len(acc))
print("Models:", model)
print("Accuracies:", acc)
```

```
Length of model: 4
Length of acc: 4
Models: ['Decision Tree', 'Naive Bayes', 'RF', 'Logistic Regression']
Accuracies: [0.972972972972973, 0.6936936936936937, 1.0, 0.8648648648648649]
```

```
# Build dataframe for plotting
results_df = pd.DataFrame({
    "Algorithm": model,
    "Accuracy": acc
})

plt.figure(figsize=(10,5), dpi=100)
sns.barplot(data=results_df, x="Accuracy", y="Algorithm", palette="dark")
plt.title("Accuracy Comparison")
plt.xlabel("Accuracy")
plt.ylabel("Algorithm")
plt.show()
```



```
#pickling the file
import pickle
pickle_out = open('classifier.pkl','wb')
pickle.dump(RF,pickle_out)
pickle_out.close()
```

Start coding or [generate](#) with AI.

```
from google.colab import files
files.download("classifier.pkl")
files.download("fertilizer.pkl")
```

```
model = pickle.load(open('classifier.pkl','rb'))
```

```
[ ] model = pickle.load(open('classifier.pkl','rb'))
model.predict([[34,67,62,0,1,7,0,30]])
```

↔ array([7])

```
[ ] model = pickle.load(open('classifier.pkl','rb'))
model.predict([[25,78,43,4,1,22,26,38]])
```

↔ array([7])

```
[ ] #pickling the file
import pickle
pickle_out = open('fertilizer.pkl','wb')
pickle.dump(encode_fert,pickle_out)
pickle_out.close()
```

```
[ ] fert = pickle.load(open('fertilizer.pkl','rb'))
fert.classes_[1]
```

↔ 'Potassium chloride'

```
[ ] # testing our model

model = pickle.load(open("classifier.pkl","rb"))
y_pred = model.predict([[25,78,43,4,1,22,26,38]])

# Convert prediction back to fertilizer name
predicted_fert = fert.inverse_transform(y_pred)
print(predicted_fert[0])
```

↔ Urea + TSP + Potassium chloride

```
[ ] #testing our model

model = pickle.load(open("classifier.pkl","rb"))
y_pred = model.predict([[35,80,23,14,9,29,36,18]])

# Convert prediction back to fertilizer name
predicted_fert = fert.inverse_transform(y_pred)
print(predicted_fert[0])
```

↔ TSP